

İLERİ CSS TEKNİKLERİ



- Flexbox'a Giriş
- Basit Menü Düzeni Örnek Uygulaması
- İleri Flexbox Teknikleri
- Navigasyon Menüsü Örnek Uygulaması
- Css Grid Sistemi
- Responsive Tasarım Ve Media Queries
- Media Queries'e Giriş: Temel Söz Dizimi
- Mobile-First (Önce Mobil) Yaklaşımı: Modern Standart
- Animasyon ve Geçiş Efektleri (Animations and Transitions)
- Transition (Geçiş Efektleri)
- CSS Keyframe Animasyonları: Tam Kontrol
- Genel Uygulama: Kişisel Portfolyo Kartları

İÇİNDEKİLER



- Bu üniteyi çalıştıktan sonra;
 - Flexbox sistemini kullanarak web sayfalarında esnek düzenler kurabileceksiniz.
 - CSS Grid ile satır ve sütun yapıları oluşturarak gelişmiş sayfa düzenleri tasarlayabileceksiniz.
 - Responsive (Duyarlı) tasarımın temel ilkelerini kavrayıp mobil, tablet ve masaüstü için uyumlu sayfalar geliştirebileceksiniz.
 - Media queries kullanarak farklı ekran boyutlarına özel stiller yazabileceksiniz.
 - Transition ve transform özellikleriyle öğelere basit geçiş ve görsel efektler ekleyebileceksiniz.
 - @keyframes kullanarak animasyonlar tanımlayacak ve kullanıcı etkileşimlerini zenginleştirebileceksiniz.
 - Tüm bu teknikleri bir arada kullanarak modern, duyarlı ve etkileşimli bir web sayfası oluşturabileceksiniz.

HEDEFLER



Atatürk Üniversitesi
Açıköğretim Fakültesi

WEB TASARIMININ TEMELLERİ Doç. Dr. Yusuf İslam BOLAT

ÜNİTE 7

GİRİŞ

Web teknolojilerinin hızla gelişmesiyle birlikte, kullanıcıların beklentileri de aynı ölçüde artmaktadır. Günümüzde yalnızca doğru çalışan bir web sitesi yeterli görülmemekte; aynı zamanda estetik, esnek, cihazlara uyumlu ve etkileşimli tasarımlar ön plana çıkmaktadır. Bu noktada CSS'in gelişmiş özellikleri, web tasarımcılarına ve geliştiricilere güçlü araçlar sunar. Bu kitap bölümünün amacı, öğrencilerin temel CSS bilgisini bir adım öteye taşıyarak modern web tasarımında kullanılan ileri teknikleri kavramalarını sağlamaktır.

Bu bölümde öncelikle *Flexbox ve CSS Grid* sistemleriyle düzen oluşturma mantığı ele alınacaktır. Öğrenci, bu araçların yalnızca birer alternatif değil, belirli ihtiyaçlara uygun güçlü çözümler sunduğunu görecektir. Flexbox'ın esnek yapısı sayesinde küçük ölçekli düzenleri kolayca kurabilecekken, Grid sistemi ile daha karmaşık sayfa tasarımlarını yönetebilecektir. Bu sayede, öğrenci sayfa düzenlerinde yalnızca klasik yöntemlere bağlı kalmadan daha modern ve profesyonel çözümler üretebilecektir.

Ardından, günümüz web tasarımında vazgeçilmez bir konu olan *responsive tasarım* üzerinde durulacaktır. Farklı boyutlardaki cihazlarda, aynı web sitesinin kullanıcıya doğru şekilde görünmesi ve işlevselliğini koruması büyük önem taşır. Öğrenci, media queries kullanarak farklı ekran genişliklerine özel tasarımlar yapmayı, mobil öncelikli yaklaşımı benimsemeyi ve modern CSS fonksiyonlarıyla (örneğin min(), max() ve clamp()) tasarımları daha esnek hale getirmeyi öğrenecektir. Böylece, yalnızca masaüstü değil, telefon ve tablet kullanıcılarına da etkili bir deneyim sunabilme becerisi kazanacaktır.

Bölümün ilerleyen kısımlarında ise *CSS geçişleri ve animasyonları* ele alınacaktır. Öğrenci, transition ve transform özellikleriyle temel görsel efektler ekleyebilecek, ardından @keyframes ile daha gelişmiş hareketli tasarımlar oluşturabilecektir. Bu bölüm yalnızca görsel estetiğe değil, aynı zamanda kullanıcı deneyimini geliştiren küçük etkileşimlere de odaklanacaktır. Çünkü modern web sitelerinde animasyonlar yalnızca göze hitap eden süslemeler değil, aynı zamanda kullanıcıyı yönlendiren, bilgilendiren ve deneyimi zenginleştiren araçlardır.

Son olarak, öğrenci tüm bu teknikleri bir araya getirerek *tamamen duyarlı ve etkileşimli bir web sayfası* geliştirme sürecine hazırlanacaktır. Bu, yalnızca tek tek konuları öğrenmekle kalmayıp onları bir bütün içerisinde uygulayabilme yetkinliği kazandıracaktır. Bölümün sonunda önerilen final proje, öğrenciye gerçek dünyada karşılaşılabileceği bir web tasarımı deneyimi sunacak ve kendi portfolyosuna ekleyebileceği somut bir ürün geliştirmesine olanak tanıyacaktır.

Bu bölüm, öğrencilere yalnızca teknik beceri kazandırmayı değil, aynı zamanda modern web tasarımının mantığını anlamalarını da amaçlamaktadır. Böylece öğrenci, öğrenme sürecini tamamladığında hem bireysel çalışmalarında hem de profesyonel ortamlarda kullanabileceği güncel ve ileri düzey CSS yetkinliklerine sahip olacaktır.

FLEXBOX'A GİRİŞ

Web sayfalarında düzen kurarken geleneksel yöntemler olan *float*, *inline-block* veya *position* gibi özellikler uzun yıllar kullanılmıştır. Ancak bu yöntemler karmaşık düzenlerde hem zaman alıcı olmuş hem de çoğu zaman beklenen esnekliği sağlayamamıştır. İşte bu noktada *Flexbox (Flexible Box Layout)* sistemi modern CSS ile birlikte hayatımıza girmiş ve düzen kurmayı hem daha kolay hem de daha güçlü hale getirmiştir.

Tablo 7.1. Temel Flexbox Özellikleri

Özellik	Açıklama
display: flex	Bir öğeyi Flex Container (esnek kapsayıcı) haline getirir.
flex-direction	Öğelerin sıralanma yönünü belirler (row, column, row-reverse, vb.).
justify-content	Öğeleri yatay eksende hizalar (center, space-between, vb.).
align-items	Öğeleri dikey eksende hizalar (center, flex-start, vb.).
flex	Öğelerin esneklik oranını belirler (flex: 1 öğeyi eşit yayar).

Flex Container ve Flex Item Kavramı

Flexbox kullanabilmek için öncelikle bir *container (kapsayıcı)* ve onun içerisindeki *item (öğeler)* mantığını anlamak gerekir.

- **Container:** Flex özelliklerini taşıyan kapsayıcı öğedir.
- **Item:** Container içinde yer alan her bir alt öğedir.

Bir öğeyi Flex container'a dönüştürmek için CSS kodları içerisine aşağıdaki kod yazılır. Bu satır yazıldığında ".container" içerisindeki tüm öğeler otomatik olarak *flex item* haline gelir.

Örnek CSS kodu:

```
.container {
  display: flex;
}
```

Eksen Mantığı (Main Axis ve Cross Axis)

Flexbox'ın en önemli farkı, öğeleri bir eksen üzerinde hizalamasıdır.

- **Main axis (ana eksen):** Öğelerin yerleştirildiği temel eksendir. Varsayılan olarak soldan sağdır.
- **Cross axis (yan eksen):** Ana eksenin dik doğrultusudur.

Bu eksen mantığını anlamak, hizalama özelliklerini kullanmayı kolaylaştırır.

Temel Flexbox Özellikleri

Flexbox, bir konteyner içindeki öğeleri yatay veya dikey eksende esnek biçimde düzenlemeye olanak tanır. Karmaşık sayfa düzenlerini daha kolay ve

okunabilir kodlarla oluşturur. Özellikle responsive (duyarlı) tasarımlar için idealdir çünkü ekran boyutuna göre öğelerin konumunu ve boyutunu otomatik olarak ayarlayabilir.

Flex-direction

Container içerisindeki öğelerin ana ekseninde nasıl sıralanacağını belirler: Burada aşağıdaki seçeneklere göre öğelerin sırası değiştirilebilir.

- *row*: Soldan sağa
- *row-reverse*: Sağdan sola
- *column*: Yukarıdan aşağı
- *column-reverse*: Aşağıdan yukarı

Örnek CSS kodu:

```
.container {  
  display: flex;  
  flex-direction: row; /* varsayılan değer */  
}
```

Justify-content

Justify-content Öğelerin ana ekseninde hizalanmasını sağlar. Flex-start, flex-end, center, space-between, space-around değerlerini alır. Aldığı değerlere göre:

- *flex-start*: Başa hizalar (varsayılan)
- *flex-end*: Sona hizalar
- *center*: Ortalar
- *space-between*: Öğeler arası eşit boşluk
- *space-around*: Kenarlar ve aralar eşit boşluk

Örnek CSS kodu:

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
}
```

Align-items

Align-items Öğelerin yan ekseninde hizalanmasını sağlar: Align-items; flex-start, flex-end, center ve stretch değerlerini alabilir. Buna göre:

- *flex-start*: Üste hizalar
- *flex-end*: Alta hizalar
- *center*: Ortalar
- *stretch*: Yüksekliği kapsayıcıya yayar

Örnek CSS kodu:

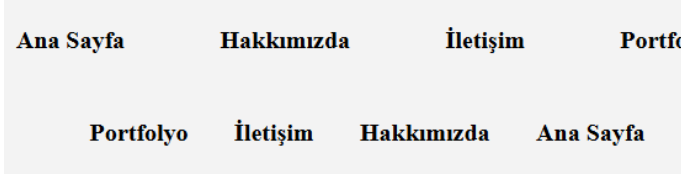
```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
}
```

```
align-items: center;
}
```

Basit Menü Düzeni Örnek Uygulaması

Aşağıdaki örnekte, dört menü öğesini öğrendiğimiz CSS kodlarıyla yatayda ortalayalım. Burada yukarıda bahsedilen fonksiyonların alabileceği değerler değiştirilerek yapacağı etki görülebilir. Aşağıdaki örnekte yer alan görselin **1. satırında** görüldüğü üzere “flex-direction: row;” ve “justify-content: space-between” kodu kullanıldığında Anasayfa Hakkımızda İletişim ve Portfolyo sırası ile ve aralarında eşit boşluklar verilerek öğelerin yerleştiği görülebilir. Bu kod parçasını “flex-direction: row-reverse;” ve “justify-content: center” olarak değiştirdiğimizde bu sıranın ve öğeler arası boşlukların değişerek **2. Satırdaki** gibi sondan başlayarak başa doğru sıralandığı görülebilir.

Tablo 7.2. Basit menü düzeni için örnek kod ve ekran görüntüsü

Html kodları:	Kodların Web Sayfasındaki Karşılığı:
<pre><div class="container"> <div>Ana Sayfa</div> <div>Hakkımızda</d iv> <div>İletişim</div> <div>Portfolyo</div> > </div></pre>	
<pre>CSS kodları: .container { display: flex; flex-direction: row; justify-content: space between; align-items: center; background-color: #f3f3f3; height: 60px; } .container div { margin: 0 15px; font-weight: bold; }</pre>	

İLERİ FLEXBOX TEKNİKLERİ

Flexbox'un temel özelliklerini öğrendikten sonra artık daha gelişmiş düzenler oluşturmak mümkündür. Bu bölümde *esneklik özellikleri, öğelerin boyutlarını yönetme ve sıralarını değiştirme* konularına odaklanacağız. Bu sayede öğrenciler, web sayfalarında daha dinamik ve profesyonel tasarımlar oluşturabileceklerdir.

Esneklik Özellikleri

Flexbox, öğelerin *genişliğini ve yüksekliğini esnek şekilde yönetebilme* yeteneği sunar. Bunu sağlayan flex-grow, flexshrink ve flex-basis olmak üzere üç temel özellik vardır. Bunların kullanım ve örnek kodları aşağıda sunulmuştur.

Flex-grow

Öğenin, container'da kalan boşluğu ne kadar büyüterek kaplayacağını belirler. Varsayılan değeri 0'dır. Bu koşulda Boşluk paylaşılmaz. 1 ve üzeri değerler alındığında ise boşluk eşit veya oranlı olarak paylaşılır.

Örnek CSS kodu:

```
.item {  
  flex-grow: 1; /* Diğer öğelerle birlikte boşluğu eşit paylaşır */  
}
```

Flex-shrink

Öğenin, container daraldığında küçülme oranını belirler. Varsayılan değeri 1'dir. Eğer 1 değerini almışsa küçülmeye izin verir. Ancak eğer 0 değerini almışsa küçülmez.

Örnek CSS kodu:

```
.item {  
  flex-shrink: 1; /* Daralmaya izin verir */  
}
```

Flex-basis

Öğenin başlangıç boyutunu belirler (piksel, yüzde veya otomatik). Flex-basis "auto" değerini alırsa içerik genişliğine göre boyutunu alır, belirli bir değer atanırsa ise sabit başlangıç boyutu alır.

Örnek CSS kodu:

```
.item {  
  flex-basis: 200px; /* Başlangıç genişliği 200px */  
}
```

Öğelerin Sırasını Değiştirme (order)

Flexbox, öğelerin HTML'deki sırasına bağlı kalmadan *ekran üzerinde farklı sıralama yapabilme* imkânı sunar. Varsayılan değeri 0'dır. Aldıkları "order" değerlerine göre item'lerden daha yüksek değer alan daha sonra yerleştirilir.

Örnek CSS kodu:

```
.item:first-child {
```

```

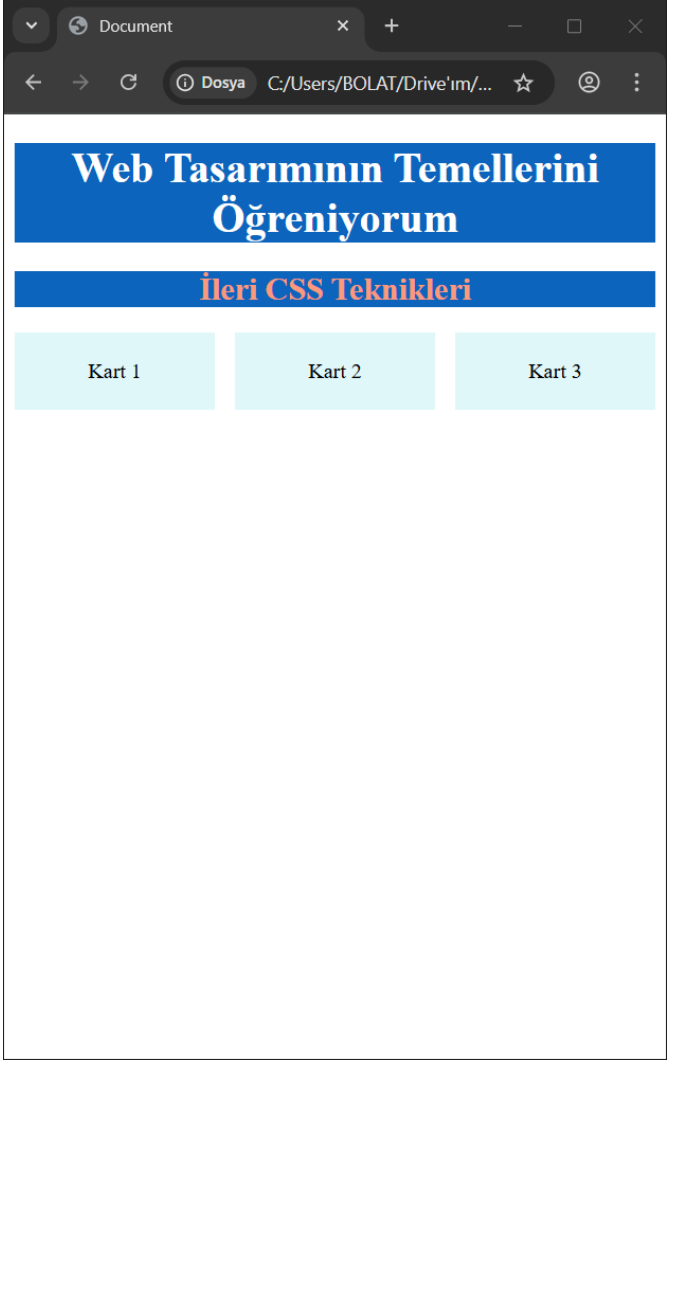
    order: 2;
}
.item:last-child {
    order: 1;
}

```

Esnek Kart Düzeni Örnek Uygulaması

Bu örnekte, Sayfaya daha önceki bölümlerden öğrendiğimiz birkaç unsuru da kullanarak *h1* ve *h2* düzeyinde başlıklar eklenmiştir. Bunun yanı sıra bu bölümün konusunu ele aldığımız flex yapısı için üç ürün kartını esnek ve ekran genişliğine uyumlu şekilde aşağıdaki gibi yerleştirilmiştir. Tablo 1.3'te kod ve sayfa görüntüsü sunulmuştur.

Tablo 7.3. Esnek kart düzeni örnek kod ve ekran görüntüsü

Html Kodları: <pre> <main> <section> <h1>Web Tasarımının Temellerini Öğreniyorum</h1> </section> <section> <h2>İleri CSS Teknikleri</h2> </section> <div class="container"> <div class="item">Kart 1</div> <div class="item">Kart 2</div> <div class="item">Kart 3</div> </div> </main> </pre> CSS Kodları: <pre> section {background- color: #0c64bc; text-align: center;} h1{color: #fff;} h2{color: #fc997c;} .container { display: flex; justify-content: space- between; align-items: stretch; gap: 15px; /* Öğeler arası boşluk */ } </pre>	
---	---

```
.item {  
    flex-grow: 1; /*  
    Kartlar boşluğu eşit  
    paylaşır */  
    flex-basis: 150px; /*  
    Başlangıç genişliği */  
    padding: 20px;  
    background-color:  
    #e0f7fa;  
    text-align: center;  
}
```

Navigasyon Menüsü Örnek Uygulaması

Flex yapısını kalıcı bir şekilde öğrenebilmek adına biraz daha gelişmiş bir örnek uygulama ile devam etmek önemli görülmektedir. Bu örnek uygulamada bir navigasyon menüsü oluşturup öğeler ekran boyutuna göre esnek şekilde yerleşmesini sağlanmıştır. Anasayfa ve İletişim öğelerinin sırası **order** ile değiştirilerek farklı bir görsel düzen oluşturulmuştur. Tablo 1.4'te kod ve sayfa görüntüsü sunulmuştur. Tablo 1.4'te order ile navigasyon öğelerine müdahale edilerek sıralaması değiştirilmiştir. Html kodlarına göre navigasyon öğelerinin Anasayfa, Hakkında, Hizmetler, İletişim sırası ile gitmesi gerekirken order kullanımından sonra tabloda sunulduğu gibi sıralanmıştır. Bu örnekte de sunulduğu gibi Order kullanımında küçükten büyüğe doğru bir sıralama kullanıldığı görülmektedir.

Tablo 7.4. Navigasyon Menüsü örnek kod ve ekran görüntüsü**Html Kodları:**

```

<main>
  <section>
    <h1>Web Tasarımının
    Temellerini
    Öğreniyorum</h1>
  </section>
  <section>
    <h2>İleri CSS
    Teknikleri</h2>
  </section>
  <nav class="menu">
    <div class="item
    home">Ana Sayfa</div>
    <div class="item
    about">Hakkında</div>
    <div class="item
    services">Hizmetler</div>
    <div class="item
    contact">İletişim</div>
  </nav>
</main>

```

CSS kodları

```

.menu {
  display: flex;
  justify-content: space-
  between; /* öğeleri
  yatayda yay */
  align-items: center; /*
  Dikey ortalama */
  gap: 20px; /* Öğeler
  arası boşluk */
  background-color:
  #f5f5f5;
  padding: 15px;
  border-radius: 8px;
}
/* Menü öğeleri */
.item {
  flex-grow: 1; /*
  Boşluğu eşit paylaş */
  flex-shrink: 1; /* Ekran
  daraldığında küçülebilir */
  text-align: center;
  padding: 10px;
  background-color:
  #9fccda;
  border-radius: 5px;
}

```



```

    cursor: pointer;
  }
  /* Sıralama değişikliği */
  .home {
    order: 2; /* Ana sayfa 2.
    sıraya geçti */
  }
  .services {
    order: 4; /* Hizmetler 4.
    sıraya geçti */
  }
  .about {
    order: 3; /* Hakkında 3'e
    geldi */
  }
  .contact {
    order: 1; /* İletişim en
    başa geldi */
  }

```

CSS GRID SİSTEMİ

CSS Grid sistemi, özellikle karmaşık ve iki boyutlu düzenlerin oluşturulmasında büyük avantaj sağlar. Flexbox, tek boyutlu hizalamalar için ideal olsa da örneğin bir ürün galerisinde hem satır hem sütun düzeni gerektiğinde yetersiz kalabilir. CSS Grid ile bir e-ticaret sitesinde ürün kartlarını 4 sütun ve 3 satır şeklinde hizalamak, her bir kartın sabit boyutta kalmasını sağlamak ve boşlukları eşit şekilde dağıtmak oldukça kolaydır. Ayrıca, haber portallarında manşet haberin geniş bir alana yayılması, yanındaki diğer haberlerin daha küçük kutulara yerleştirilmesi gibi asimetrik düzenler de Grid sayesinde zahmetsizce uygulanabilir. Dashboard arayüzlerinde grafik, tablo ve menülerin belirli alanlara sabitlenmesi, ekran boyutuna göre yeniden düzenlenmesi gibi **responsive** tasarımlar da Grid'in güçlü yönlerindendir. Bu sistem, geliştiricilere hem okunabilir hem de sürdürülebilir bir kod yapısı sunarak modern web tasarımında esneklik ve kontrol sağlar. Tablo 1.5'te aralarındaki fark özetlenmiştir.

Tablo 7.5. Grid ve Flexbox karşılaştırması

Özellik	CSS Grid	Flexbox
Düzen Tipi	İki boyutlu (satır ve sütun)	Tek boyutlu (ya satır ya da sütun)
Kullanım Amacı	Karmaşık layout'lar, sayfa iskeleti	İçerik hizalama, küçük bileşen düzenlemeleri
Yerleşim Kontrolü	Satır ve sütunları aynı anda kontrol eder	Sadece bir eksenle kontrol sağlar.
Öğe Konumlandırma	Belirli hücreye doğrudan yerleştirme yapılabilir.	Sıralı yerleşim, sıraya göre konumlandırır.
Responsive Tasarım	Alanları kolayca yeniden düzenleyebilir.	İçerik akışına göre esnek davranır.

Her iki yönteminde kullanım amaçlarına göre güçlü yanları bulunmaktadır. Ne Zaman Hangisi Kullanılır. CSS Grid sistemi; Dashboard, galeri, haber portalı gibi karmaşık sayfa düzenleri, satır ve sütunların birlikte kontrol edilmesi gereken durumlar, belirli alanlara içerik yerleştirme ihtiyacı varsa kullanılması daha uygundur. Flexbox; navigasyon menüsü, buton grupları, kart listeleri gibi tek eksenli düzenler, içeriklerin dinamik olarak hizalanması gereken durumlar, küçük bileşenlerin responsive davranışları için kullanımı daha uygundur. Bir HTML elemanını grid konteyneri yapmak için tek yapmanız gereken ona *display: grid;* özelliğini vermektir. Bunun için bir container'ı Grid'e dönüştürmek için aşağıdaki kod eklenmelidir:

Örnek CSS kodu:

```
.grid-container {  
  display: grid;  
}
```

Bu kodlar ile henüz görsel olarak bir değişiklik olmamıştır. Ancak .grid-container artık bir grid sisteminin kurallarını uygulamaya hazır hale gelmiştir. İçindeki öğeler (.grid-item) artık birer grid öğesine dönüşmüştür. Bu kod ile grid-container içerisindeki tüm öğeler Grid hücrelerine yerleşir. Grid'in üç ana bileşeni aşağıda sunulmuştur:

- *Grid Container* → *display: grid* tanımlanan kapsayıcı
- *Grid Items* → *Container* içindeki öğeler
- *Satırlar ve sütunlar* → *Grid* düzenini oluşturan yapı taşları

Grid sisteminin en önemli özelliği yapıyı satır ve sütunlara bölmek olarak belirtmiştik. Bu sistemde *grid-template-columns* ile sütunların genişliğini ve sayısını, *grid-template-rows* ile ise satırların yüksekliğini ve sayısını belirleriz. Öğelerimizi 3 sütun ve 2 satırdan oluşan bir ızgaraya yerleştirdik. Her hücre 150x100 piksel boyutunda yerleştirmek için aşağıdaki kodu kullanabiliriz.

Örnek CSS kodu:

```
.grid-container {  
  display: grid;  
  /* 3 adet sütun oluştur. Her biri 150 piksel genişliğinde olsun. */  
  grid-template-columns: 150px 150px 150px;  
  /* 2 adet satır oluştur. Her biri 100 piksel yüksekliğinde olsun. */  
  grid-template-rows: 100px 100px;  
}
```

Esnek Birim: fr (Fraction- Kesir)

Grid sisteminde sabit bir değer vermek yerine daha esnek bir kullanımda mümkündür. Bu işlem için "fr" birimi kullanılabilir. Fr birimi ile piksel gibi sabit değerler yerine, mevcut alanı orantılı olarak bölüşmek çok daha esnek bir kullanım sağlanabilir. 1fr, kullanılabilir alanın 1 parçasını temsil eder. Aşağıdaki kod ile container'ın genişliği ne olursa olsun her zaman 3 eşit parçaya bölmüş oluyoruz.

Örnek CSS kodu:

```
.grid-container {
```

```
display: grid;
/* Mevcut alanı 3 eşit parçaya (sütuna) böl. */
grid-template-columns: 1fr 1fr 1fr;
grid-template-rows: 100px 100px;
}
```

Eğer herhangi bir parçanın daha yüksek bir orana sahip olmasını istiyorsak kolon oranını aşağıdaki gibi değiştirebiliriz. Bu sayede 1. ve 3. parçalar bir birimlik bir genişliğe sahipken 2. Birim diğerlerinin iki katı boyutlara sahip olacaktır.

```
/* 1. sütun toplam alanın 1 payını, 2. sütun 2 payını, 3. sütun 1 payını alır. */
grid-template-columns: 1fr 2fr 1fr;
```

Gridler’de Boşluk Kullanımı

Gridlerde öğeler arası boşluk kullanımı için 2 farklı yöntem vardır. İlk olarak gap ile tek bir ölçü ile standart bir boşluk ayarlanabilir. Gap hem satır hem sütun boşluğunu ayarlar. İkinci olarak da sütun ve satırlar arası boşlukları ayrı ayrı ayarlamaktır. Bu seçenekte column-gap sadece sütunlar arası boşluğu ayarlarken, row-gap: sadece satırlar arası boşluğu ayarlar.

Örnek CSS kodu:

```
.grid-container {
display: grid;
grid-template-columns: 1fr 1fr 1fr;
grid-template-rows: 100px 100px;
/* Tüm hücreler arasında 15 piksel boşluk bırak. */
gap: 15px; }
```

Grid *gap* özelliğini kullanarak tüm öğelerin arasında hem yatayda hem de dikeyde 15 piksellik bir boşluk oluşturuldu. Bu yöntem, *margin* yapısını kullanmaktan çok daha pratik bir çözüm sunmaktadır.

Gridler’de Öğe Yerleştirme

Öğelerin grid içerisinde nereye ve ne kadar alan kaplayacağını belirleyebiliriz. Bunu yapılandırmak için *Grid Çizgileri* mantığını bilinmesi gerekmektedir. Bir önceki örnek CSS kodumuzda 3 sütunluk bir grid yapısı hazırlamıştık. Buna göre Şekil-1.1’de görüldüğü gibi 3 sütunlu bir grid’de 4 dikey çizgi vardır.



Şekil 7.1. Üç Grid Yapısında Sütun Çizgileri

Grid yapısında, grid-column-start / grid-column-end: Öğenin hangi dikey çizgiden başlayıp hangisinde biteceğini belirlerken grid-row-start / grid-row-end: Öğenin hangi yatay çizgiden başlayıp hangisinde biteceğini gösterir. Tablo 1.5’te öğelerin yerleştirilmesine ilişkin örnek kodlar ve sayfa yapısı sunulmuştur.

Tablo 7.6 Gridler’de Öğe Yerleştirme Örnek Uygulaması

Html Kodları:

```

<main>
  <section>
    <h1>Web
    Tasarımının Temellerini
    Öğreniyorum</h1>
  </section>
  <section>
    <h2>İleri CSS
    Teknikleri</h2>
  </section>
  <div class="grid-
  container">
    <div class="grid-item
  item-1">Öğe 1</div>
    <div class="grid-item
  item-2">Öğe 2</div>
    <div class="grid-item
  item-3">Öğe 3</div>
    <div class="grid-item
  item-4">Öğe 4</div>
    <div class="grid-item
  item-5">Öğe 5</div>
    <div class="grid-item
  item-6">Öğe 6</div>
  </div>
</main>

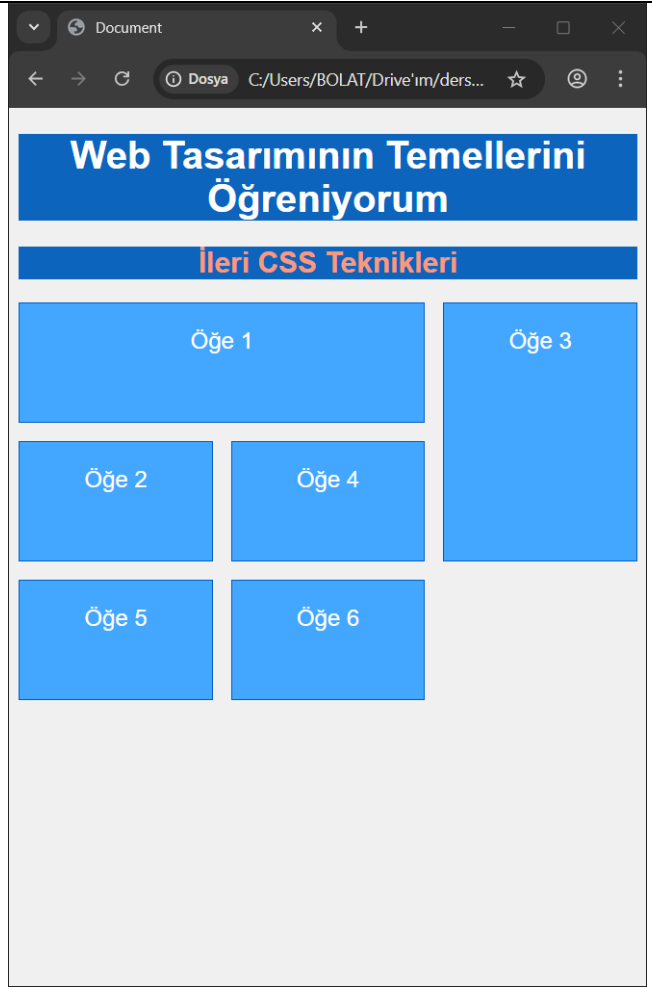
```

CSS kodları:

```

.grid-container {
  display: grid;
  grid-template-
  columns: 1fr 1fr 1fr;
  grid-template-rows:
  100px 100px 100px; /*
  3 satır yapalım */
  gap: 15px;
}
.item-1 {
  /* 1. sütun
  çizgisinden başla, 3.
  sütun çizgisine kadar
  uzan. (2 sütun kapla) */
  grid-column-start: 1;
  grid-column-end: 3;
  /* 1. satırda kal. */
  grid-row-start: 1;

```



```
    grid-row-end: 2;
}
.item-3 {
    /* 3. sütun
    çizgisinden başla, 4.
    çizgide bit. */
    grid-column-start: 3;
    grid-column-end: 4;
    /* 1. satır çizgisinden
    başla, 3. satır çizgisine
    kadar uzan. (2 satır
    kapla) */
    grid-row-start: 1;
    grid-row-end: 3;
}
.grid-item {
    background-color:
#44a7ff;
    color: white;
    border: 1px solid
#005cbf;
    padding: 20px;
    font-size: 1.2em;
    text-align: center; }
```

Bu kod yapısını biraz daha kısaltmak için başlangıç ve bitiş arasında “/” kullanımı da bulunmaktadır. Örneğin 1. sütun çizgisinden başlayıp 3. Sütun çizgisine kadar alanı kaplaması için `grid-column-start: 1; grid-column-end: 3;` yapısını kullanmak yerine `grid-column: 1 / 3;` kullanılabilir. Bunun yanı sıra *span* anahtar kelimesi de kullanılabilir. Bunun için ise `grid-column: 1 / span 2;` kod parçası kullanılabilir. Bu kodda span anahtar kelimesi kaplayacağı alanı ifade eder, yani 1. çizgiden başla ve 2 sütunluk yer kapla anlamına gelmektedir. Bunun yanı sıra işlemleri hızlandırmak için **repeat** anahtar kelimesinden de yararlanılabilir. Repeat Karmaşık düzenlerde tekrar eden satır/sütun tanımları için kullanılır. Örnek kullanımı aşağıdaki gibidir:

Örnek CSS kodu:

```
grid-template-columns: repeat(3, 1fr); /* 3 sütun, her biri eşit genişlik */

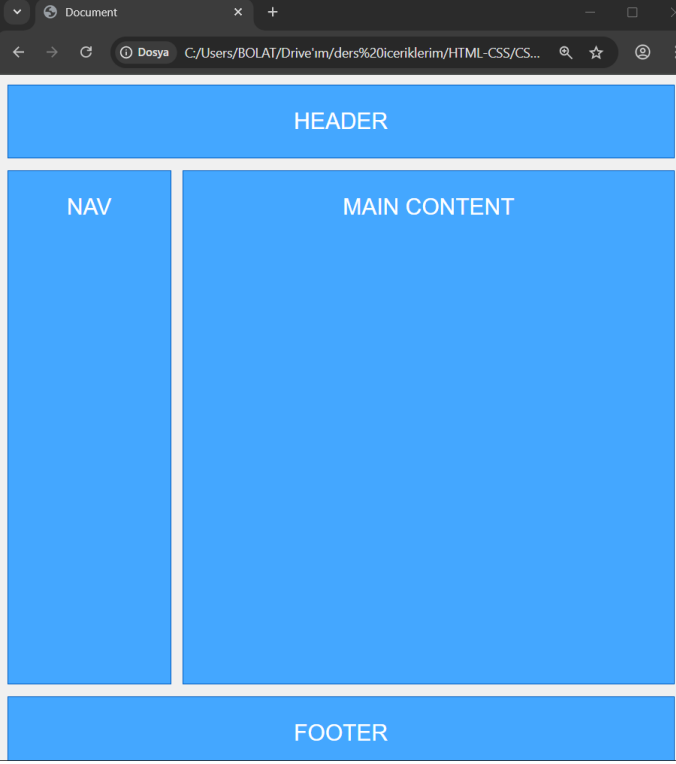
grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
/* sütunlar minimum 150px olur, yer kaldıkça genişler.*/
```

Grid’lerde İsimlendirilmiş Alanlar

Bu, grid’in en sezgisel ve güçlü özelliklerinden biridir. Sayfa düzenini bir metin gibi çizerek oluşturmanızı sağlar. İlk olarak Öğelere İsim Ver: grid-area

özellikleri ile her öğeye bir isim verilir. İkinci adımda Şablonu Çiz: grid-template-areas ile konteyner içinde bu isimleri kullanarak bir şablon oluşturulur. Tablo 1.7’de Gridler’de isimlendirilmiş alan örnek uygulaması sunulmuştur. grid-template-areas içindeki yapı, grid’in görsel halini temsil eder. Buna göre ilk satırın tamamını (header header) "header" isimli alan kaplar. İkinci satırın ilk sütununu (nav), ikinci sütununu ise (main) ilgili alanlar kaplar. Üçüncü satırın tamamını (footer footer) "footer" alanı kaplar. Bu yöntem, özellikle sayfa ana iskeletini oluştururken okunaklı ve yönetilebilir yapı oluşturmayı sağlar.

Tablo 7.7 Gridler’de İsimlendirilmiş Alan Örnek Uygulaması

Html kodları: <pre> <main> <div class="grid- container"> <header class="grid- item">HEADER</heade r> <nav class="grid- item">NAV</nav> <main class="grid- item">MAIN CONTENT</main> <footer class="grid- item">FOOTER</footer > </div> </pre> CSS kodları: <pre> /* Önce her alana bir isim verelim */ header { grid-area: header; } nav { grid-area: nav; } main { grid-area: main; } footer { grid-area: footer; } .grid-container { display: grid; grid-template- columns: 1fr 3fr; /* Sol bar ve ana içerik için sütunlar */ grid-template-rows: auto 1fr auto; /* Header, içerik, footer için satırlar */ gap: 10px; </pre>	 The screenshot shows a web browser window with a document titled 'Document'. The address bar shows the file path 'C:/Users/BOLAT/Drive'im/ders%20iceriklerim/HTML-CSS/CS...'. The browser displays a grid layout with four sections: a blue 'HEADER' bar at the top, a blue 'NAV' sidebar on the left, a large blue 'MAIN CONTENT' area in the center, and a blue 'FOOTER' bar at the bottom.
--	--

```
height: 100vh; /*
Sayfa yüksekliği kadar
olsun */
/* Şimdi şablonu
çiziyoruz */
grid-template-areas:
  "header header"
  "nav main"
  "footer footer";
}
.grid-item {
  background-color:
#44a7ff;
  color: white;
  border: 1px solid
#005cbf;
  padding: 20px;
  font-size: 1.2em;
  text-align: center;
}
```

RESPONSİVE TASARIM VE MEDIA QUERİES

Günümüzde web sitelerine sadece bilgisayarlardan değil, çok çeşitli ekran boyutlarına sahip telefon, tablet ve hatta televizyonlardan erişilmektedir. Responsive Tasarım (Türkçe: Duyarlı Tasarım), bir web sitesinin, içinde görüntülediği cihazın ekran boyutuna ve özelliklerine otomatik olarak uyum sağlaması prensibidir. Böylece her kullanıcı için en iyi deneyimi sunulması amaçlanmaktadır. Bunu yapmamızı sağlayan en güçlü araç ise Media Queries'dir.

Responsive tasarımı için tek bir kod tabanı ile her ekranda uyumlu görünüm sağlamak olarak da tanımlayabiliriz. Responsive tasarım, kullanıcı deneyimini iyileştirir (UX), tüm cihazlarda tutarlı bir görünüm sağlar, ayrı bir mobil siteye gerek kalmaz ve SEO açısından Google tarafından önerilir. Responsive tasarıma başlamadan önce HTML dosyanızın <head> etiketleri arasına mutlaka eklemeniz gereken hayati bir satır vardır:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Bu satırda `width=device-width` Tarayıcıya, sayfanın genişliğini cihazın kendi ekran genişliği olarak ayarlamasını söyler. Bu, mobil cihazların sitenizi "küçültülmüş bir masaüstü sitesi" gibi göstermesini engeller. Diğer bir kod parçası `initial-scale=1.0` ise sayfa ilk yüklendiğinde herhangi bir yakınlaştırma (zoom) yapılmamasını sağlar. Bu adımı atlamazsanız, tüm responsive çabalarınızın boşa gitmesine neden olabilir. ***Bu yüzden her projenizin başında bu etiket eklenmelidir.***

Media Queries'e Giriş: Temel Söz Dizimi

Media Query (Medya Sorgusu), belirli koşullar sağlandığında CSS kurallarını uygulamamızı sağlayan bir yapıdır. Yani CSS içinde belirli koşullara göre farklı stiller

uygulamamızı sağlar. En yaygın kullanılan koşul ekran genişliğidir. Söz dizimi şöyledir:

```
@media medya-türü and (koşul) {  
    /* Koşul sağlandığında uygulanacak CSS kuralları buraya yazılır. */  
    .selector {  
        property: value;  
    }  
}
```

Buradaki söz dizimini biraz daha anlaşılır hale getirmek için tanımlamalar yapmak faydalı olacaktır.

- **@media:** Bir medya sorgusu başlattığımızı belirtir.
- **medya-türü:** Genellikle screen (ekranlar için), print (yazdırma için) veya all (tümü için) kullanılır. Modern web geliştirmede neredeyse her zaman screen veya all kullanılır.
- **and:** Birden fazla koşulu birleştirmek için kullanılır (isteğe bağlı).
- **(koşul):** En önemli kısımdır. Genellikle ekran genişliği ile ilgili koşullar belirtilir. En yaygın olanları:
 - **max-width:** "Ekran genişliği en fazla X piksel ise" anlamına gelir.
 - **min-width:** "Ekran genişliği en az X piksel ise" anlamına gelir.

Örnek CSS kodu:

```
body {  
    background-color: lightblue; /* Varsayılan arka plan rengi */  
}  
/* Ekran genişliği en fazla 600px ise aşağıdaki kuralları uygula */  
@media screen and (max-width: 600px) {  
    body {  
        background-color: lightcoral;  
    }  
}
```

Yukarıdaki kodu önceki projeleriniz içerisinde CSS içerisine yapıştırıp, tarayıcıda pencere boyutunu daraltıp, genişlik 600 pikselin altına düştüğü anda arka plan renginin değiştiği görülebilir. Burada @media query koşul ifadesi kullanılarak başlangıçta arka plan rengi "lightblue" iken sayfa genişliği **max-width: 600px** koşulu sağlandığında arka plan rengi "lightcoral" olarak değişmektedir.

Mobile-First (Önce Mobil) Yaklaşımı: Modern Standart

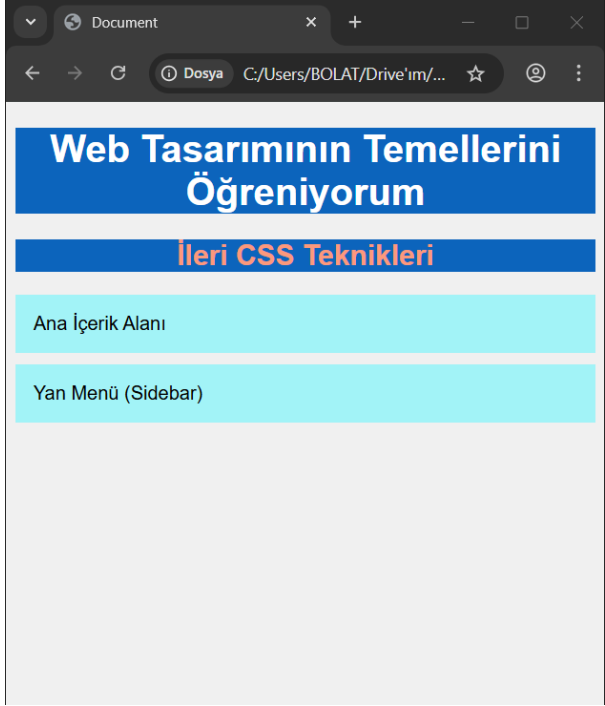
Responsive tasarım yapmanın iki yaygın yolu vardır: Masaüstünden başlayıp mobile doğru küçültmek (Desktop-First) veya mobilden başlayıp masaüstüne doğru genişletmek (Mobile-First). Mobile-First, günümüzün en iyi pratiği olarak kabul edilir. Bu durumun nedenleri aşağıdaki gibi sıralanabilir:

- **Performans:** Mobil cihazlar genellikle daha yavaş internet bağlantılarına sahiptir. Önce mobil için temel ve basit stilleri yüklemek, sitenin daha hızlı açılmasını sağlar.
- **Odaklanma:** Önce küçük ekran için tasarım yapmak, sizi en önemli içerik ve işlevlere odaklanmaya zorlar.

- **Daha Temiz Kod:** Genellikle daha az kod yazmayı ve var olan kuralları ezmek (override) yerine yeni kurallar eklemeyi sağlar.
- **Mobile-First** yaklaşımında min-width kullanılır. Bu yaklaşımın temelindeki mantık "Varsayılan stiller mobil içindir, ekran büyüdükçe yeni stiller ekle." şeklinde açıklanabilir.

Ana içerik alanı ve yan menü içeren iki sütunlu bir yapıyı responsive hale getirmek için örnek bir kod yazalım. Bu örnekte, küçük ekranlarda (768px altı) içerik ve yan menü alt alta sıralanır. Ekran 768px genişliğine ulaştığı anda, **display: flex** devreye girer ve yan yana iki sütunlu bir yapıya dönüşürler. Bu, responsive tasarımın en temel ve güçlü uygulama şeklidir. Tablo 1.8’de kod ve sayfa yapısı sunulmuştur. Bu örnekte sayfa yapısı 768 pikselden daha küçük bir genişliğe getirilerek ekran alıntısı yapılmıştır.

Tablo 7.8 Responsive Tasarım Örnek Uygulaması

Html Kodları: <pre> <section> <h1>Web Tasarımının Temellerini Öğreniyorum</h1> </section> <section> <h2>İleri CSS Teknikleri</h2> </section> <div class="container"> <div class="main- content">Ana İçerik Alanı</div> <aside class="sidebar">Yan Menü (Sidebar)</aside> </div> CSS kodları: /* --- Mobil Stilleri (Varsayılan) --- */ .container { width: 100%; } .main-content, .sidebar { background-color: #a2f3f7; padding: 15px; margin-bottom: 10px; } /* --- Tablet ve Üzeri Stiller (Ekran 768px ve daha genişse) --- */ @media screen and (min- width: 768px) { .container { </pre>	
---	---

```
display: flex; /* Sütunları
yan yana getirmek için
Flexbox kullanalım */
gap: 20px;
}
.main-content {
flex: 3; /* Ana içerik 3
birim yer kaplasın */
margin-bottom: 0;
}
.sidebar {
flex: 1; /* Yan menü 1
birim yer kaplasın */
}
}
```

İkinci bir örnek ile konunun pekişmesi ve bilgisayar, tablet ve mobil cihazlara ayrı ayrı tasarım planı için uygulama yapılması faydalı olacaktır. Bilgisayarda 4 sütun, tablet ekranında 2 sütun, telefon ekranında ise tek sütun yapısına sahip bir sayfa tasarımı hazırlanacaktır. Tablo 1.9’da ilgili kod ve sayfa görüntüsü sunulmuştur. Sayfa genişliği 480 pikselden büyük tutularak ekran alıntısı yapılmış tablet görüntüsüne ilişkin şartlardaki sayfa yapısı gösterilmiştir.

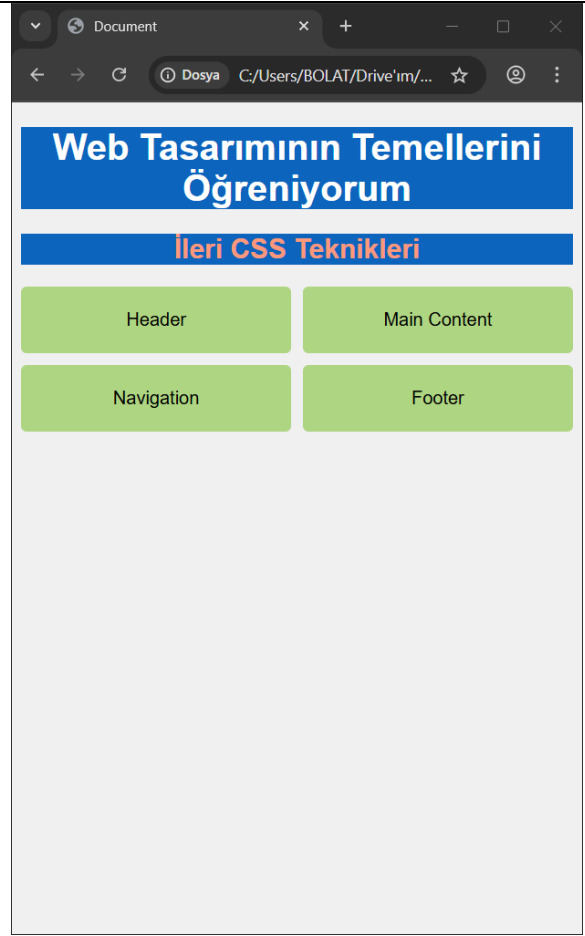
Tablo 7.9 Responsive Tasarım Örnek Uygulaması-2

Html Kodları:

```
<div class="grid">
  <div
class="item">Header</div>
  <div class="item">Main
Content</div>
  <div
class="item">Navigation</div>
>
  <div
class="item">Footer</div>
</div>
```

CSS kodları:

```
.grid {
  display: grid;
  grid-template-columns:
repeat(4, 1fr); /* 4 sütun */
  gap: 10px; }
.item {
  background: #aed581;
  padding: 20px;
  text-align: center;
  border-radius: 5px;
}
/* Tablet görünümü (max
768px) → 2 sütun */
@media (max-width: 768px) {
  .grid {
    grid-template-columns:
repeat(2, 1fr);
  }
}
/* Mobil görünümü (max
480px) → 1 sütun */
@media (max-width: 480px) {
  .grid {
    grid-template-columns:
1fr; }
}
```



Animasyon ve Geçiş Efektleri (Animations and Transitions)

Web sitelerine hayat ve profesyonel bir dokunuş katan en önemli unsurlardan biri de animasyonlardır. CSS, JavaScript'e ihtiyaç duymadan, sadece birkaç satır kodla ilgi çekici, akıcı ve performanslı animasyonlar oluşturmamıza olanak tanır. Bu işlemler için **transition** ve **animation** olmak üzere iki temel yöntemimiz bulunmaktadır.

Transition (Geçiş Efektleri)

Transition, bir CSS özelliğinin değerinin **zaman içinde yumuşak bir şekilde değişmesini** sağlar. Burada Hangi CSS özelliğinin değişeceğini, ne kadar sürede değişeceğini ve geçiş efektinin hız eğrisini tanımlanması gerekmektedir. Temel CSS Söz dizimi aşağıdaki gibidir:

```
.element {  
  transition: özellik süre kolaylaştırma-geçiş;  
}
```

Basit bir örnek uygulama ile Transition konusunu somutlaştıralım. Bir butonumuz olsun ve üzerine imlecin getirildiğinde renk değişimi gerçekleşecektir. Ancak butonun üzerine fare ile gelindiğinde renk bir anda değişmez, **yarım saniyede yumuşak bir şekilde** dönüşür.

Html Kodu:

```
<button class="btn">Tıkla</button>
```

CSS Kodu:

```
.btn {  
  background-color: #4caf50;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 5px;  
  transition: background-color 0.5s ease;  
}  
.btn:hover {  
  background-color: #2e7d32;  
}
```

CSS Keyframe Animasyonları: Tam Kontrol

Transition ile sadece başlangıç ve bitiş durumu arasındaki geçişi kontrol ederken, @keyframes ile bir animasyonun ara adımları da tanımlanarak çok daha karmaşık ve döngüsel animasyonlar oluşturulabilir. Keyframe ile animasyon yapımında süreç iki adımdan oluşur. **Birincisi @keyframes** kuralı ile animasyon senaryosunu (adımlarını) tanımlamaktır. İkinci adım ise **animation** özelliklerini kullanarak bu senaryoyu bir elemente uygulamaktır.



animation-fill-mode: forwards; burada çok önemlidir. Bu özellik olmazsa, animasyon bittiği anda element başlangıçtaki (görünmez) haline geri döner. forwards diyerek

Örnek

- Bir "içeri süzülme" animasyonu tanımlayalım.
- 1. adımda Animasyonun zaman içindeki adımlarını yüzde (%) veya from/to anahtar kelimeleriyle tanımlarız slideInFromLeft isminde bir @keyframe tanımlası yaparız. 2. adımda ise "Yeni Mesajınız Var!" ifadesini içeren bir Div yapısına uygulayacağımızı seçer ve bunun nasıl uygulanacağını ise **animation-name**: slideInFromLeft ile tanımlamış olduğumuz animasyonu, süresini, tekrarlama durumunu, animasyon bittiğinde olacak koşulları ve hız eğrisi ile ifade ederiz.

Tablo 7.10 Animasyon Örnek Uygulaması

1. adım CSS kodları:

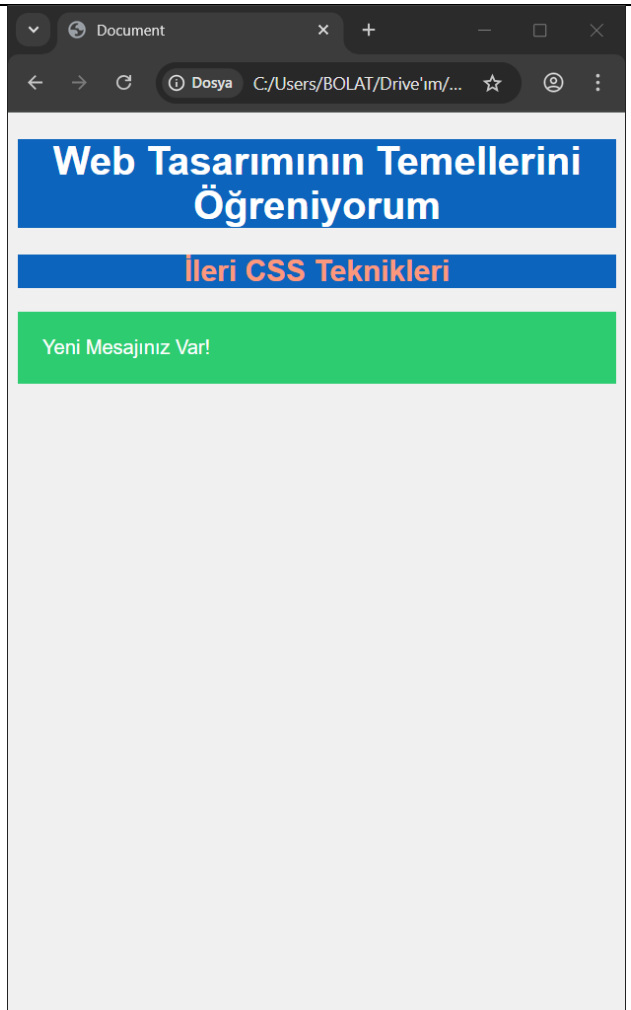
```
@keyframes
slideInFromLeft {
  /* Animasyonun
başlangıcında (%0) */
  from {
    /* Elementi ekranın
sol dışına taşı ve şeffaf
yap */
    transform:
translateX(-100%);
    opacity: 0;
  }

  /* Animasyonun
sonunda (%100) */
  to {
    /* Elementi orijinal
konumuna geri getir ve
görünür yap */
    transform:
translateX(0);
    opacity: 1;
  }
}
```

2. Adım

Html Kodları:

```
<div
class="notification">Yeni
Mesajınız Var!</div>
```





Animation özelliklerini
tek satırda
yazabilirsiniz:
animation:
slideDownFromLeft 0.8s
ease-out 1 forwards;

CSS kodları:

```
.notification {  
    padding: 20px;  
    background-color:  
#2ecc71;  
    color: white;  
/* ANİMASYONU  
UYGULA */  
    animation-name:  
slideDownFromLeft; /* Hangi  
@keyframes  
kullanılacak? */  
    animation-duration:  
0.8s; /* Animasyon  
ne kadar sürecek? */  
    animation-timing-  
function: ease-out; /* Hız  
eğrisi */  
    animation-iteration-  
count: 1; /* Kaç kez  
tekrarlanacak? (infinite:  
sonsuz) */  
    animation-fill-mode:  
forwards; /* Animasyon  
bitince son halinde mi  
kalsın? */  
}
```



Animasyon yaparken width, height, margin, left, top gibi özellikler canlandırmaktan kaçının. Bu özellikler tarayıcının sayfa düzenini (layout) yeniden hesaplamasına neden olur ve performansı düşürebilir. Bunun yerine, her zaman **transform** (translateX, translateY, scale, rotate) ve **opacity** özelliklerini canlandırmaya öncelik verin. Bu iki özellik, tarayıcı tarafından çok daha verimli bir şekilde işlenir ve bu da daha akıcı, takılmayan animasyonlar anlamına gelir.



Örnek

•Bir "Etkileşimli Kart Tasarımı": Bir ürün kartı tasarlayacağız. Fare üzerine geldiğinde kartın hafifçe yukarı kalkmasını (transform: translateY(-10px);) ve hafif bir gölge kazanmasını (box-shadow) transition kullanarak gerçekleştireceğiz.

Tablo 7.11. Animasyon Örnek Uygulaması-2

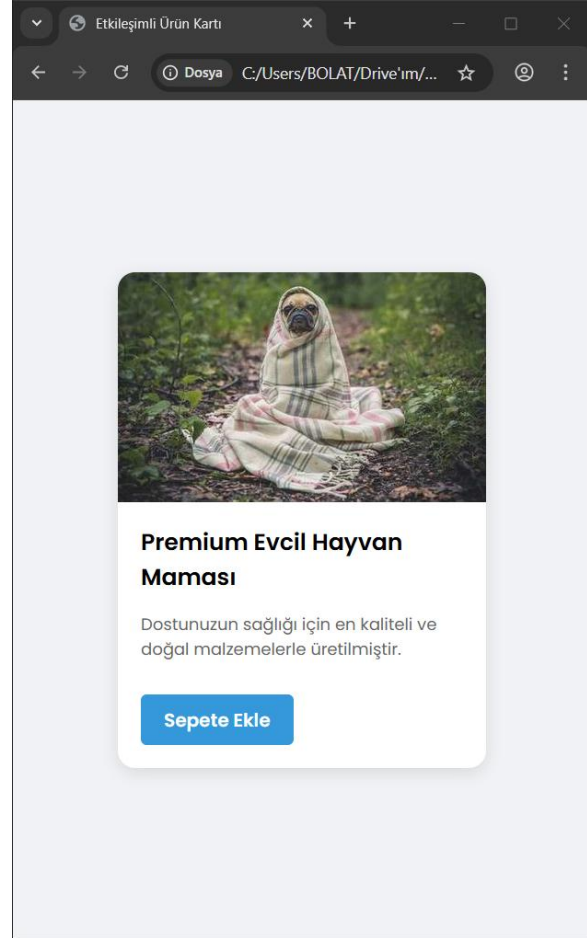
Html Kodları:

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">

  <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>Etkileşimli Ürün Kartı</title>
  <link rel="stylesheet"
href="style_Animation.css">
  <link rel="preconnect"
href="https://fonts.googleapis.com">
  <link rel="preconnect"
href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=
Poppins:wght@400;600&display=swap"
rel="stylesheet">
</head>
<body>

  <div class="product-card">
    
    <div class="card-content">
      <h3>Premium Evcil Hayvan Maması</h3>
      <p>Dostunuzun sağlığı için en kaliteli ve
doğal malzemelerle üretilmiştir.</p>
      <a href="#" class="card-button">Sepete
Ekle</a>
    </div>
  </div>
</body>
</html>
```

CSS kodları:




```
/* Sayfanın genel ayarları ve kartı ortalamak için
*/
body {
    font-family: 'Poppins', sans-serif;
    background-color: #f0f2f5;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    margin: 0;
}

/* --- Ürün Kartının Stilleri --- */

/* 1. KARTIN NORMAL (VARSAYILAN) DURUMU
*/
.product-card {
    width: 320px;
    background-color: white;
    border-radius: 15px; /* Modern bir görünüm
için köşeleri yuvarlatalım */
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1); /*
Başlangıçta hafif bir gölge */
    overflow: hidden; /* Resmin yuvarlak
köşelerden taşmasını engeller */

    /* SİHİRLİ KISIM: transition
    Hangi özelliklerin, ne kadar sürede ve hangi
    hız eğrisiyle değişeceğini belirtiyoruz.
    'transform' ve 'box-shadow' özelliklerinin
    değişimini 0.3 saniyede 'ease' efektiyle yumuşat.
    */

    transition: transform 0.3s ease, box-shadow
0.3s ease;
}

/* 2. FARE KARTIN ÜZERİNE GELDİĞİNDEKİ
DURUM (:hover) */
.product-card:hover {
    /* Kartı 10 piksel yukarı kaldır */
    transform: translateY(-10px);

    /* Gölgeyi daha belirgin hale getirerek
    "kalkma" hissini güçlendir */
    box-shadow: 0 12px 24px rgba(0, 0, 0, 0.2);
}

/* --- Kartın İçeriğinin Stilleri (Görsel
Güzelleştirme) --- */
```

```
.product-card img {  
    width: 100%;  
    display: block; /* Resmin altındaki istenmeyen  
    boşluğu kaldırır */  
}  
.card-content {  
    padding: 20px;  
}  
.card-content h3 {  
    margin-top: 0;  
    margin-bottom: 10px;  
    font-size: 1.25rem;  
}  
.card-content p {  
    font-size: 0.9rem;  
    color: #666;  
    line-height: 1.5;  
}  
  
.card-button {  
    display: inline-block;  
    margin-top: 15px;  
    padding: 10px 20px;  
    background-color: #3498db;  
    color: white;  
    text-decoration: none;  
    border-radius: 5px;  
    font-weight: 600;  
    text-align: center;  
}
```

Sonuç olarak transition ve animation kullanım alanları ve şekilleri duruma göre tercih sebebi olabilir. İki animasyon türü arasındaki farkları Tablo 1.12’ de sunulmuştur.

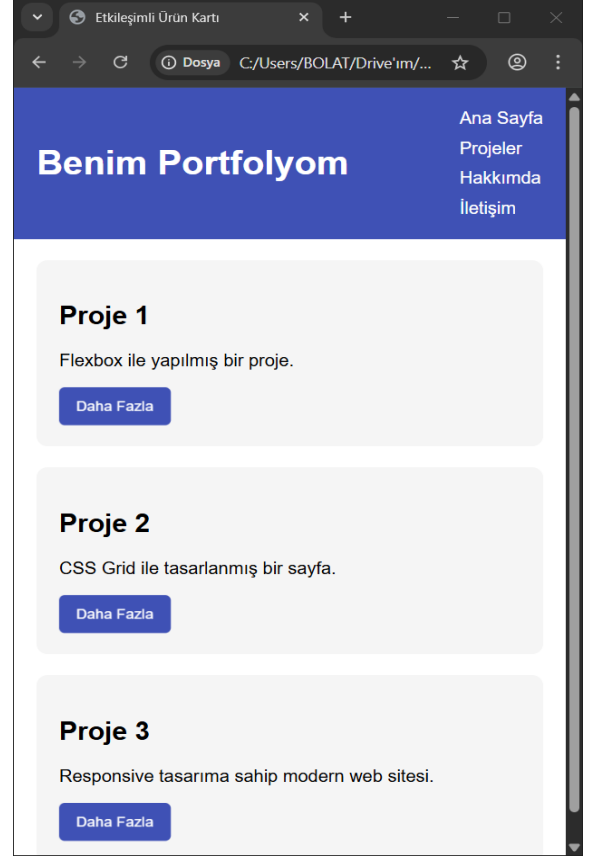
Özellik	Transition	Animation
Başlangıç	Kullanıcının etkileşimiyle (hover, click) başlar	Sayfa yüklenince otomatik başlayabilir
Hareket Türü	Başlangıç ve bitiş arasında geçiş	Birden fazla aşama tanımlanabilir

Özellik	Transition	Animation
Tekrarlama	Tekrar etmez	infinite ile sınırsız tekrar edebilir

Genel Uygulama: Kişisel Portfolyo Kartları

Html Kodu:

```
<div class="container">
  <header class="header">
    <h1>Benim Portfolyom</h1>
    <nav class="menu">
      <a href="#">Ana Sayfa</a>
      <a href="#">Projeler</a>
      <a href="#">Hakkımda</a>
      <a href="#">İletişim</a>
    </nav>
  </header>
  <main class="grid">
    <div class="card">
      <h2>Proje 1</h2>
      <p>Flexbox ile yapılmış bir proje.</p>
      <button>Daha Fazla</button>
    </div>
    <div class="card">
      <h2>Proje 2</h2>
      <p>CSS Grid ile tasarlanmış bir sayfa.</p>
      <button>Daha Fazla</button>
    </div>
    <div class="card">
      <h2>Proje 3</h2>
      <p>Responsive tasarıma sahip modern web sitesi.</p>
      <button>Daha Fazla</button>
    </div>
  </main>
</div>
```



```
font-family: Arial, sans-serif;
margin: 0;
padding: 0;
}
/* Header kısmı */
.header {
background: #3f51b5;
color: white;
padding: 20px;
display: flex;
justify-content: space-between;
align-items: center;
}

.menu {
display: flex;
gap: 20px; /* Flexbox gap */
}

.menu a {
color: white;
text-decoration: none;
transition: color 0.3s ease; /*
Transition */
}

.menu a:hover {
color: #ffeb3b;
}

/* Grid sistemi ile kartlar */
.grid {
display: grid;
grid-template-columns:
repeat(3, 1fr);
gap: 20px;
padding: 20px;
}

.card {
background: #f5f5f5;
padding: 20px;
border-radius: 10px;
transition: transform 0.5s ease;
/* Kart hover efekti */
}

.card:hover {
transform: scale(1.05);
}

/* Buton animasyonu */
button {
background: #3f51b5;
```

```
color: white;
padding: 10px 15px;
border: none;
border-radius: 5px;
cursor: pointer;
animation: pulse 2s infinite; /*
Animation */
}
button:hover {
background: #283593;
}
/* Animasyon tanımı */
@keyframes pulse {
0% { transform: scale(1); }
50% { transform: scale(1.1); }
100% { transform: scale(1); }
}
/* Responsive tasarım */
@media (max-width: 768px) {
.grid {
grid-template-columns: 1fr;
}
.menu {
flex-direction: column;
gap: 10px;
}
}
```



Özet

- Bu bölümde, modern web tasarımında merkezi bir yere sahip olan düzenleme, esneklik ve etkileşim kavramları bir bütünlük içerisinde ele alınmıştır. Öncelikle Flexbox sistemi, esnek kutu modeli bağlamında incelenmiş ve öğelerin hem yatay hem de dikey eksenlerde dengeli bir biçimde hizalanmasına imkân tanıyan özellikleriyle açıklanmıştır. Flexbox'ın sunduğu satır ve sütun temelli düzenleme imkânlarının, farklı ekran boyutlarına uyum sağlamadaki avantajları vurgulanmıştır. Özellikle menü yapılarında ve temel içerik düzenlemelerinde esnekliğin kullanıcı deneyimine doğrudan katkı sağladığı ifade edilmiştir. Bu yönüyle Flexbox, çağdaş web tasarımında yalnızca teknik bir kolaylık değil, aynı zamanda estetik bütünlüğü ve işlevsel tutarlılığı destekleyen temel bir araç olarak değerlendirilmiştir.
- CSS Grid sistemi ise bölümün ikinci önemli bileşenini oluşturmuştur. Grid, satır ve sütun yapısına dayalı matris düzenlemeleriyle içeriklerin görsel bütünlük içinde sunulmasına olanak tanımaktadır. Bu sistem, özellikle çok sayıda portfolyo kartı, galeri ögesi veya içerik kutusunun düzenli bir matris üzerinde hizalanması gerektiğinde güçlü bir çözüm sunmaktadır. Grid'in sunduğu kolon genişlikleri, satır aralıkları ve alan yerleşimlerinin çeşitliliği, tasarımcıya daha özgür ve planlı bir düzen kurma imkânı sağlamaktadır. Bölümde Grid sisteminin yalnızca sayfa düzenlemesi açısından değil, aynı zamanda bilginin kullanıcıya tutarlı ve anlaşılır biçimde sunulmasına yaptığı katkı da öne çıkarılmıştır.
- Üçüncü olarak, responsive tasarım ilkeleri ve media queries konusu ele alınmıştır. Farklı cihaz türlerinde ve ekran boyutlarında tutarlı bir kullanıcı deneyimi sunmanın, günümüz web tasarımında vazgeçilmez bir gereklilik olduğu ifade edilmiştir. Media queries sayesinde içeriklerin mobil, tablet ve masaüstü cihazlara uygun biçimde düzenlenebileceği belirtilmiştir. Bu yaklaşım, kullanıcı merkezli tasarım anlayışının temel bir uzantısı olarak görülmüş ve erişilebilirlik ile kapsayıcılığın web tasarımındaki önemine vurgu yapılmıştır. Responsive tasarım, yalnızca görsel uyum sağlamakla kalmamakta, aynı zamanda kullanıcı çeşitliliğini göz önünde bulundurarak web sitelerinin daha geniş bir kitleye hitap etmesine aracılık etmektedir.
- Transition kavramı ise kullanıcı etkileşimlerinde süreklilik, akışkanlık ve estetik bütünlük sağlamadaki rolü bağlamında değerlendirilmiştir. Özellikle hover ve tıklama gibi kullanıcı hareketlerine verilen tepkilerin yumuşak geçişlerle desteklenmesi, deneyimi profesyonel ve akıcı hâle getirmektedir. Transition, yalnızca estetik bir unsur değil, aynı zamanda kullanıcıyı yönlendiren, etkileşimi kolaylaştıran ve görsel sürekliliği sağlayan işlevsel bir tasarım ögesi olarak öne çıkarılmıştır.
- Son olarak animation teknikleri incelenmiştir. Web sayfalarına hareket ve canlılık kazandırmak amacıyla kullanılan animasyonların, estetik bir çekicilik oluşturmanın ötesinde kullanıcı dikkatini yönlendiren işlevsel araçlar olduğu vurgulanmıştır. Keyframes yapısının kullanımı, animasyon sürelerinin ayarlanması ve farklı efektlerin bütünleştirilmesi konuları üzerinde durulmuştur. Animasyonların, sayfa açılışında içeriklerin kayarak görünmesi veya menü öğelerinin dikkat çekici biçimde hareket etmesi gibi örneklerle, web tasarımına dinamizm kazandırdığı ifade edilmiştir.
- Genel olarak bu bölümde, Flexbox, Grid, responsive tasarım, transition ve animation kavramlarının birbirini tamamlayan bütünleşik unsurlar olduğu ortaya konulmuştur. Bu tekniklerin birlikte ele alınması, yalnızca teknik düzeyde değil, aynı zamanda estetik ve işlevsellik boyutlarında da güçlü, çağdaş ve kullanıcı merkezli web deneyimlerinin oluşturulmasına katkı sağlamaktadır. Böylelikle web tasarımı, yalnızca görsel bir düzenleme faaliyeti olmaktan çıkıp, kullanıcı deneyimini merkeze alan, erişilebilirliği gözetilen ve modern teknolojik standartlarla uyumlu bir bütünleşik alan olarak değerlendirilmektedir.

DEĞERLENDİRME SORULARI

1. Flexbox hangi düzenler için en uygundur?
 - a) Tablo tabanlı düzenler
 - b) İki boyutlu düzenler
 - c) Tek boyutlu düzenler
 - d) Responsive düzenler
 - e) Grid sistemleri
2. Flexbox içinde öğelerin yatay hizalanmasını hangi özellik belirler?
 - a) align-items
 - b) justify-content
 - c) flex-direction
 - d) order
 - e) align-self
3. Flexbox'ta öğelerin görsel sırasını değiştirmek için hangi özellik kullanılır?
 - a) flex-grow
 - b) justify-items
 - c) flex-shrink
 - d) order
 - e) gap
4. Bir öğenin diğerlerinden daha fazla yer kaplaması için hangi özellik kullanılır?
 - a) flex-shrink
 - b) flex-grow
 - c) align-self
 - d) justify-content
 - e) flex-basis
5. Grid sisteminde sütunları tanımlamak için kullanılan özellik hangisidir?
 - a) grid-template-rows
 - b) grid-gap
 - c) grid-template-columns
 - d) grid-area
 - e) grid-auto-flow
6. Grid'de öğeler arasındaki boşluk hangi özellik ile ayarlanır?
 - a) grid-gap
 - b) padding
 - c) margin
 - d) justify-content
 - e) align-content

7. Grid sisteminde “fr” birimi neyi ifade eder?
 - a) Piksel ölçüsü
 - b) Yüzde
 - c) Rem birimi
 - d) Em birimi
 - e) Esnek pay
8. Bir öğeyi 2 sütun boyunca genişletmek için doğru yazım hangisidir?
 - a) grid-column: 2;
 - b) grid-template: span 2;
 - c) grid-row: 2;
 - d) grid-column: span 2;
 - e) grid-auto-flow: column;
9. Media queries hangi yapı ile tanımlanır?
 - a) @font-face
 - b) @media
 - c) @keyframes
 - d) @responsive
 - e) @container
10. Küçük etkileşimlerde kullanılan geçiş efektleri hangi özellik ile yapılır?
 - a) Animation
 - b) Keyframes
 - c) @media
 - d) Transform
 - e) transition

Cevap Anahtarı

1-c, 2-b, 3-d, 4-b, 5-c, 6-a, 7-e, 8-d, 9-b, 10-e

YARARLANILAN KAYNAKLAR

- MDN Web Docs. (2025, 14 Temmuz). Basic concepts of flexbox. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox (25 September 2025 tarihinde erişilmiştir).
- W3C CSS Working Group. (2018, 19 Kasım). CSS Flexible Box Layout Module Level 1. World Wide Web Consortium (W3C). <https://www.w3.org/TR/css-flexbox-1/> (22 September 2025 tarihinde erişilmiştir).
- MDN Web Docs. (2025, 29 Ağustos). Basic concepts of grid layout. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Basic_concepts_of_grid_layout (19 September 2025 tarihinde erişilmiştir).
- W3C CSS Working Group. (2025, 26 Mart). CSS Grid Layout Module Level 1. World Wide Web Consortium (W3C). <https://www.w3.org/TR/css-grid-1/> (16 September 2025 tarihinde erişilmiştir).
- MDN Web Docs. (2025, 14 Temmuz). Using media queries. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Media_Queries/Using_media_queries (13 September 2025 tarihinde erişilmiştir).
- W3C CSS Working Group. (2021, 18 Aralık). Media Queries Level 5. World Wide Web Consortium (W3C). <https://www.w3.org/TR/mediaqueries-5/> (10 September 2025 tarihinde erişilmiştir).
- LePage, P., & Andrew, R. (t.y.). Responsive web design basics. web.dev (Chrome for Developers). <https://web.dev/responsive-web-design-basics/> (07 September 2025 tarihinde erişilmiştir).
- W3C CSS Working Group. (2023, 5 Eylül). CSS Transitions Level 2. World Wide Web Consortium (W3C). <https://www.w3.org/TR/css-transitions-2/> (04 September 2025 tarihinde erişilmiştir).
- W3C CSS Working Group. (2023, 2 Mart). CSS Animations Level 1. World Wide Web Consortium (W3C). <https://www.w3.org/TR/css-animations-1/> (01 September 2025 tarihinde erişilmiştir).
- Basques, K., & Andrew, R. (2020, 6 Ekim). How to create high-performance CSS animations. web.dev (Chrome for Developers). <https://web.dev/high-performance-css-animations/> (29 August 2025 tarihinde erişilmiştir).